

DOCUMENTO DE PROYECTO · APLICACIÓN WEB

# RUTA 18
# Sistema de Reservas de Mesas

Hamburguesas · Sushi · Platos a la carta — Bucaramanga, Santander


AUTOR
Thania Daniela Quintana Garzon

INSTITUCIÓN
Uniciencia


AÑO
2026

01 · CONTEXTO

# El problema: reservas a mano

Ruta 18 (Cra. 18 #11-34) tiene 4.5 ★ y más de 2.000 opiniones en Google. Hoy las reservas llegan por **llamada, WhatsApp o en persona** y se anotan en una libreta.

### Doble reserva

La misma mesa se entrega dos veces.

### Sin visibilidad

El cliente no sabe qué está libre.

### Carga operativa

El admin coordina todo a mano.

### Sin historial

No hay datos de demanda.

Ejemplo real

Juan pide mesa para 4 el sábado a las 7:00 p. m. Luego María pide **la misma mesa a la misma hora** y también se anota.

Resultado: dos grupos en la puerta, un cliente molesto y una mala reseña en Google.

# ¿Qué busca el sistema?

## Objetivo general

Desarrollar una aplicación web para que los clientes **reserven mesas solos**, viendo la disponibilidad en tiempo real, y que el administrador **gestione mesas, horarios, reservas y carta** de forma eficiente.

### Ver y elegir mesa

Interfaz visual con todas las mesas, su zona y capacidad.

### Reservar en 3 pasos

Fecha → hora → datos, sin llamar ni desplazarse.

### Disponibilidad

Franjas libres y ocupadas por día y por mesa.

### Panel de administración

Mesas, horarios, reservas y carta digital.

### Responsivo

Funciona en celular, tablet y computador.

### Publicado

Código en GitHub y despliegue en Vercel.

# Qué incluye y qué no

**Sí incluye**

- ✓ Módulo cliente: mesas, fecha, hora y confirmación
- ✓ Módulo administrador con acceso por contraseña
- ✓ Carta digital por categorías con fotos
- ✓ Productos destacados en el inicio
- ✓ Validaciones y mensajes claros
- ✓ Diseño responsivo (móvil primero)
- ✓ Repositorio en GitHub y despliegue en Vercel

**Fuera del alcance**

- ✗ Pagos en línea o anticipos
- ✗ Pedidos a domicilio en línea
- ✗ App móvil nativa (iOS / Android)
- ✗ Registro de clientes
- ✗ Notificaciones por correo, SMS o WhatsApp
- ✗ Reportes de ventas e inventario

**Decisión de esta versión:** es **solo frontend**. Los datos viven en listas de JavaScript y se guardan en el navegador ( `localStorage` ) en lugar de Supabase, para entregar un prototipo completo sin servidor.

# ¿Quién usa el sistema?



## Cliente

Persona que quiere una mesa. No necesita registrarse.

- Ve mesas y disponibilidad
- Consulta la carta
- Confirma su reserva



## Administrador

Encargado del restaurante. Entra con usuario y contraseña.

- Registra y edita mesas
- Define horarios
- Gestiona reservas y carta



## Almacenamiento

El navegador guarda los datos (`localStorage`). Reemplaza a Supabase en esta versión.

- Mesas y horarios
- Reservas
- Platos y fotos

# Los 14 requisitos, cumplidos

| CÓDIGO | REQUISITO · CLIENTE | |
|---|---|---|
| RF-01 | Ver las mesas con zona, capacidad y estado | ✓ |
| RF-02 | Seleccionar una mesa | ✓ |
| RF-03 | Elegir la fecha (solo días de atención) | ✓ |
| RF-04 | Ver horarios libres y ocupados | ✓ |
| RF-05 | Reservar validando la capacidad | ✓ |
| RF-06 | Comprobante de confirmación | ✓ |
| RF-07 | Consultar la carta digital | ✓ |

| CÓDIGO | REQUISITO · ADMINISTRADOR | |
|---|---|---|
| RF-08 | Acceso al panel con contraseña | ✓ |
| RF-09 | Registrar mesas | ✓ |
| RF-10 | Listar mesas | ✓ |
| RF-11 | Gestionar horarios y duración | ✓ |
| RF-12 | Ver y filtrar reservas | ✓ |
| RF-13 | Activar / desactivar mesas | ✓ |
| RF-14 | Gestionar la carta (agregar, editar, ocultar, eliminar) | ✓ |

Requisitos no funcionales atendidos: usabilidad (reserva en 3 pasos), responsividad, código separado por archivos y comentado, compatibilidad con navegadores modernos y escalabilidad (agregar mesas o platos sin tocar código).

# Cómo está construido

### 🖥️ Presentación

**HTML5 + CSS3.** `index.html` (cliente) y `admin.html` (panel). Flexbox, Grid y media queries para el diseño responsivo.

### ⚙️ Lógica

**JavaScript ES6.** `cliente.js`, `admin.js` y `ui.js`: genera los horarios, valida formularios y muestra avisos.

### 💾 Datos

**data.js + localStorage.** Cuatro listas: mesas, horarios, reservas y platos. Se sincronizan entre pestañas.

## Stack tecnológico

HTML5 · CSS3 · JavaScript ES6 · localStorage · Google Fonts · Poppins · Git + GitHub · Vercel

Sin frameworks ni dependencias: se abre en cualquier navegador.

## Flujo de publicación

Código local → **git push** → GitHub → **Vercel** despliega automáticamente cada cambio en `main`.

# Cuatro colecciones

### mesas

`id`
`nombre`
`zona` salón / terraza
`capacidad`
`activa`
`foto_url`
`creado_en`

### horarios

`id`
`dia_semana`
`hora_inicio`
`hora_fin`
`duracion_slot` (min)
`activo`

### reservas

`id` · `mesa_id`
`cliente_nombre`
`cliente_contacto`
`num_personas`
`fecha` · `hora_inicio`
`hora_fin`
`estado`

### platos

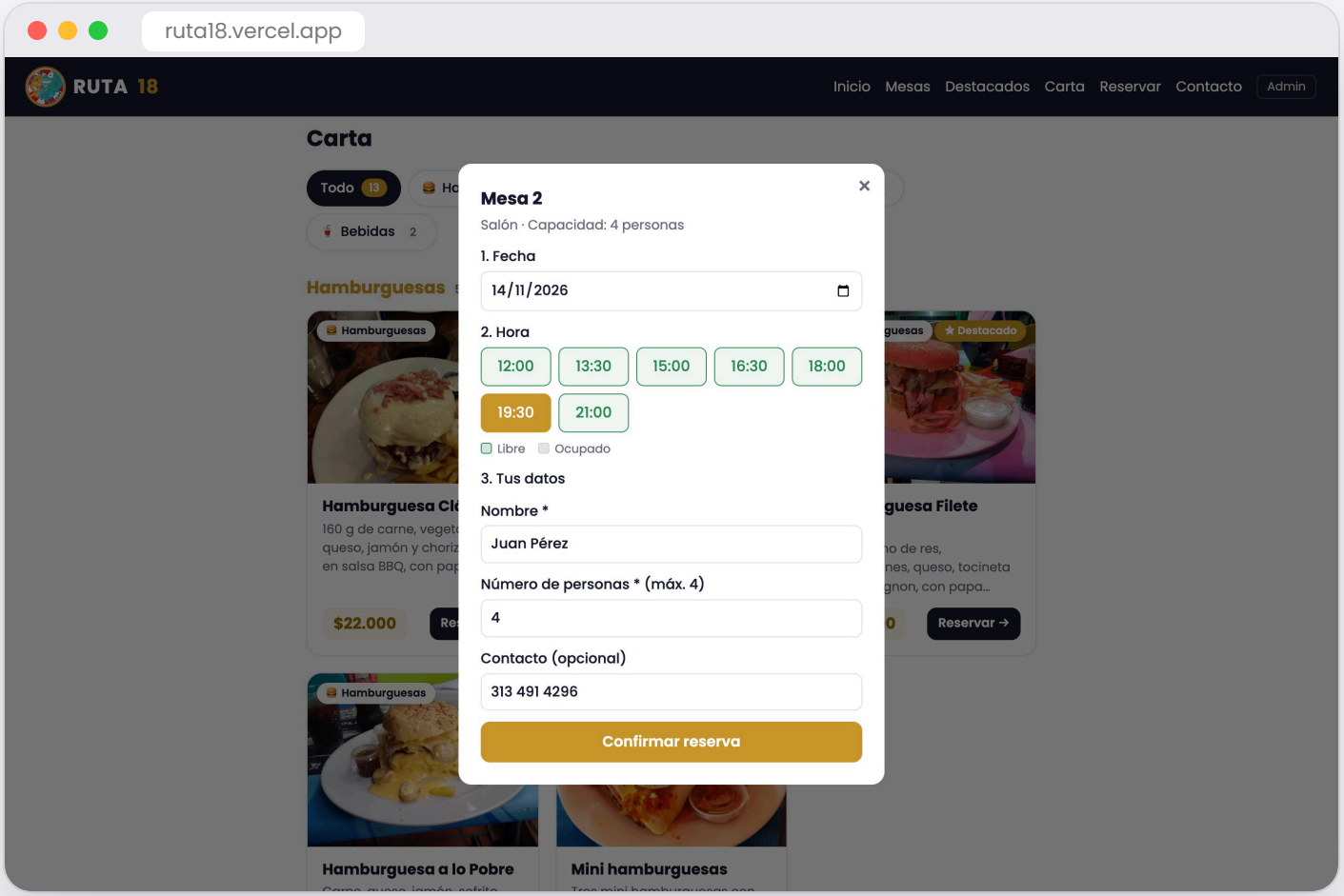
`id`
`nombre`
`categoria`
`descripcion`
`precio` (COP)
`disponible`
`destacado` · `imagen`

**Cómo se calculan los horarios:** con el horario del día y la duración (ej. 12:00–22:00, 90 min) se generan las franjas. Una franja está `ocupada` si cruza con una reserva no cancelada, y `pasada` si su hora ya pasó.

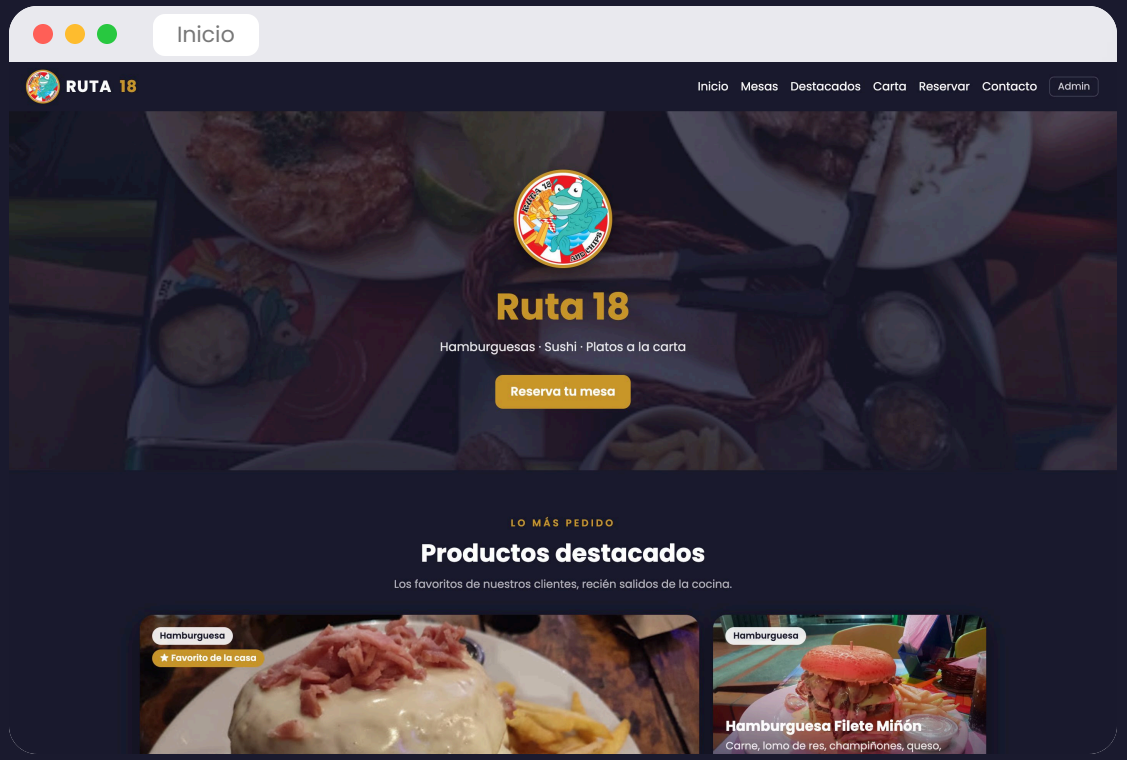
# Reservar una mesa

1. El cliente abre la página y ve la **lista de mesas**.
2. Toca una mesa y se abre el formulario de reserva.
3. Elige la **fecha**: solo días en que el restaurante atiende.
4. Elige una **hora libre** (verde). Las ocupadas o pasadas salen grises.
5. Escribe nombre, número de personas y contacto opcional.
6. Confirma: se guarda y la hora queda **ocupada**.

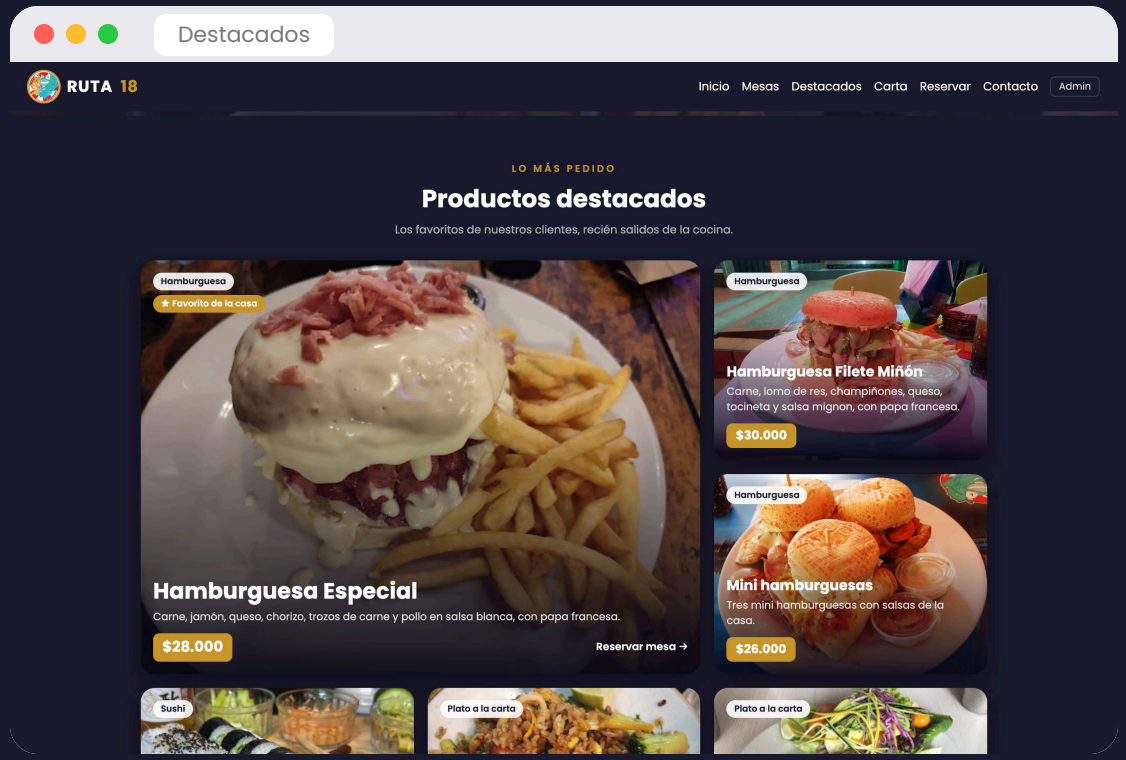
**Flujos alternativos:** hora ocupada → aviso. Más personas que la capacidad → aviso con el máximo permitido.



# Inicio y productos destacados

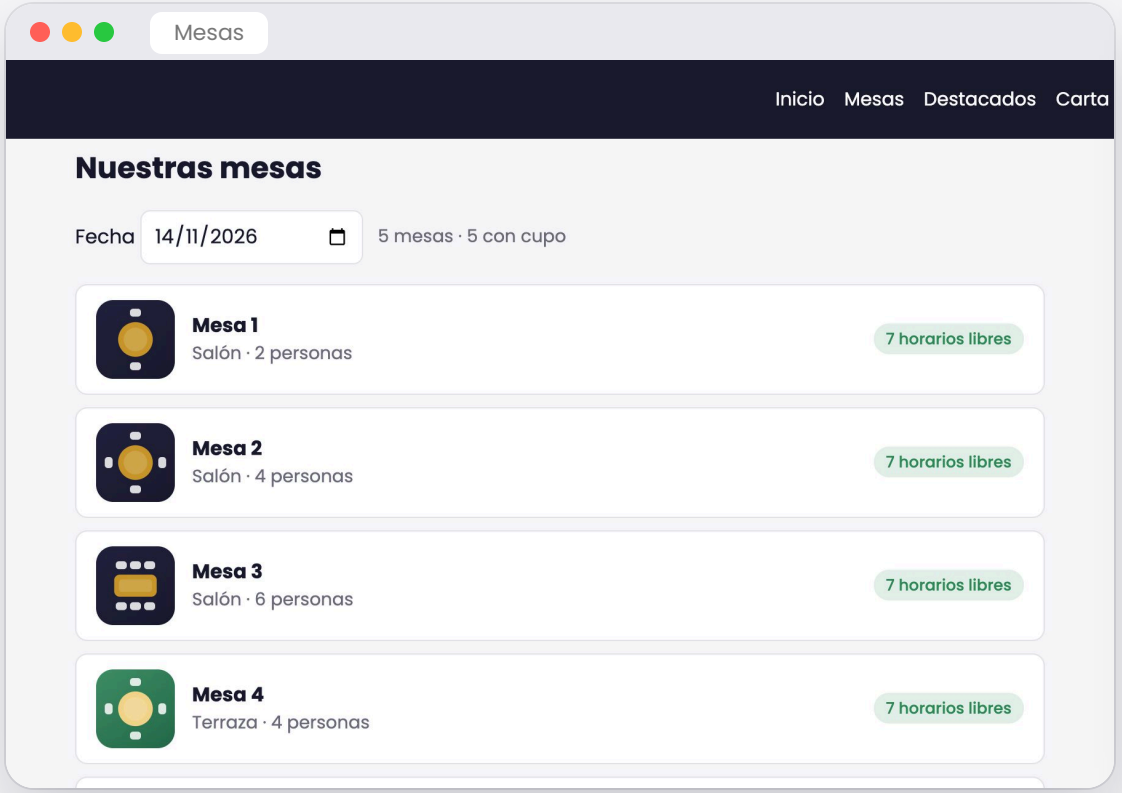



Encabezado con logo, menú y fondo con fotos reales del restaurante.

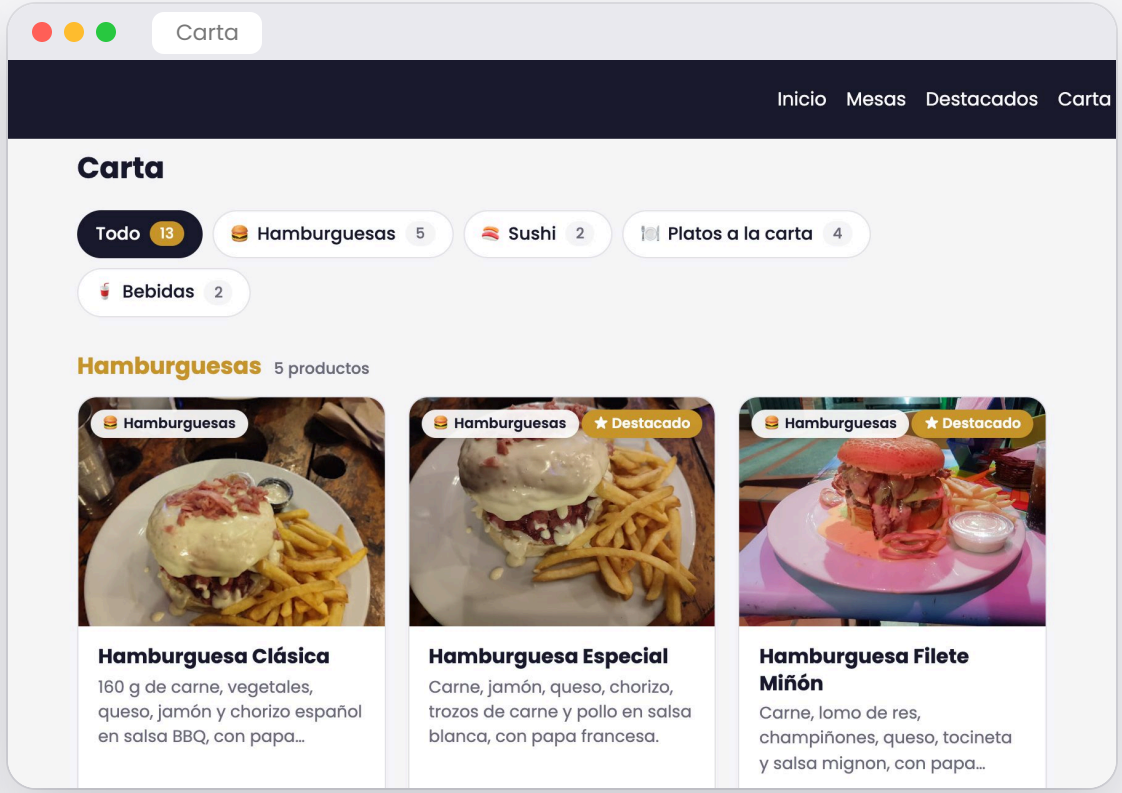



"Lo más pedido": el admin elige qué productos se muestran.

# Mesas y carta digital



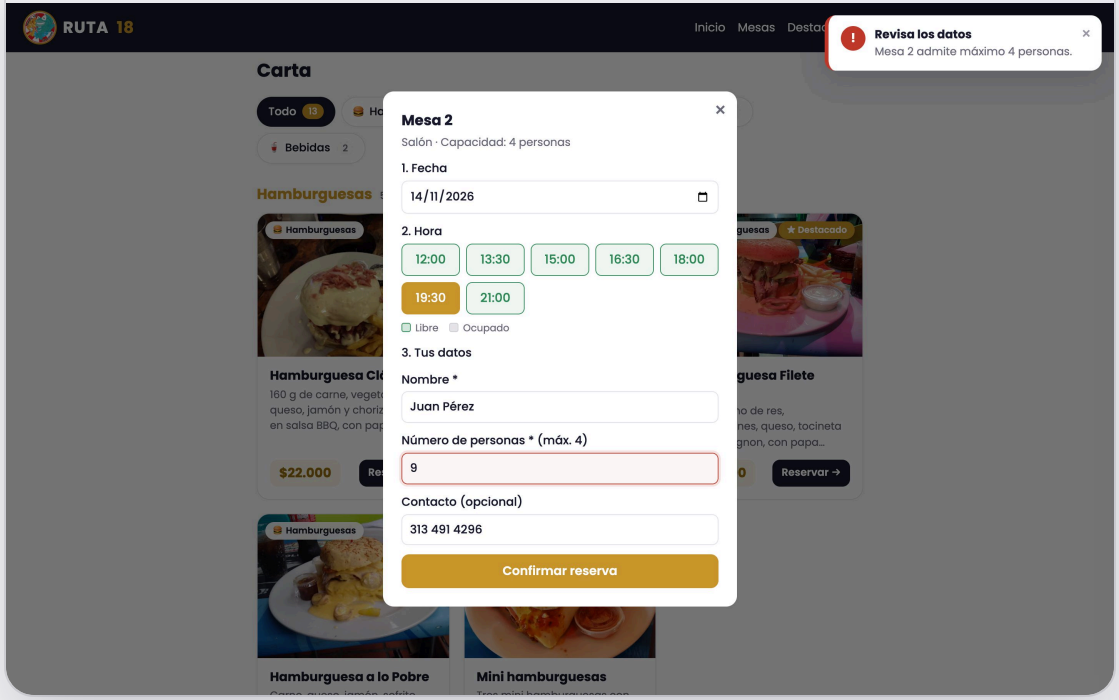

Cada mesa tiene un ícono con sus sillas reales; salón en azul y terraza en verde.




Carta en tarjetas con foto, categoría, precio y filtro por categoría.

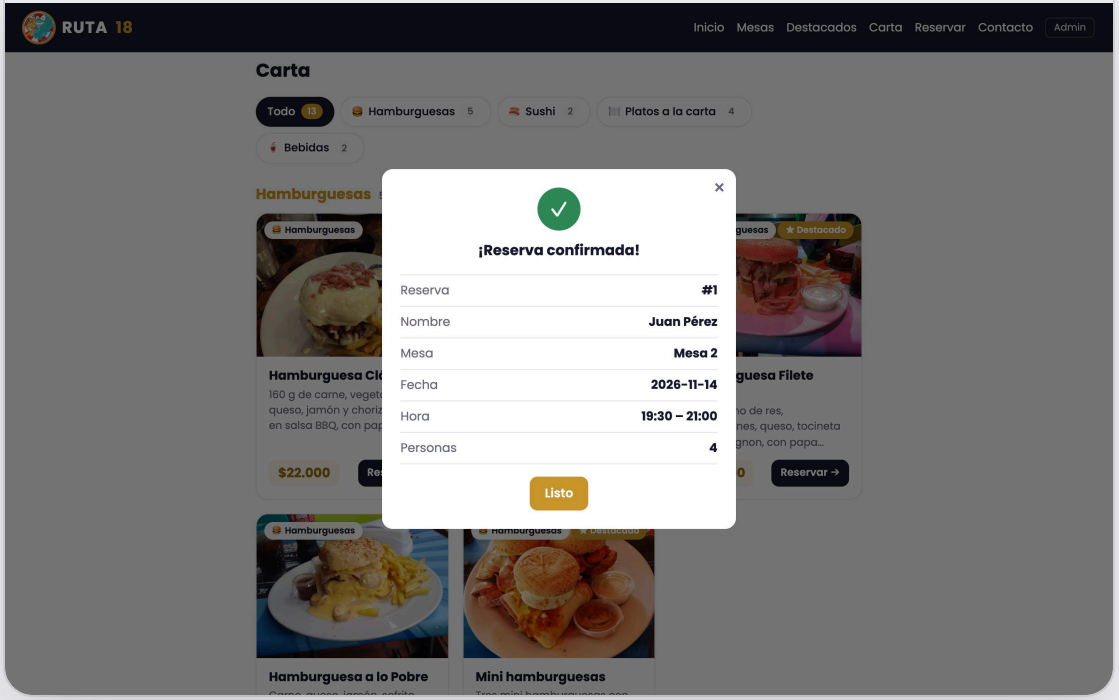
# Validaciones y confirmación

Validación



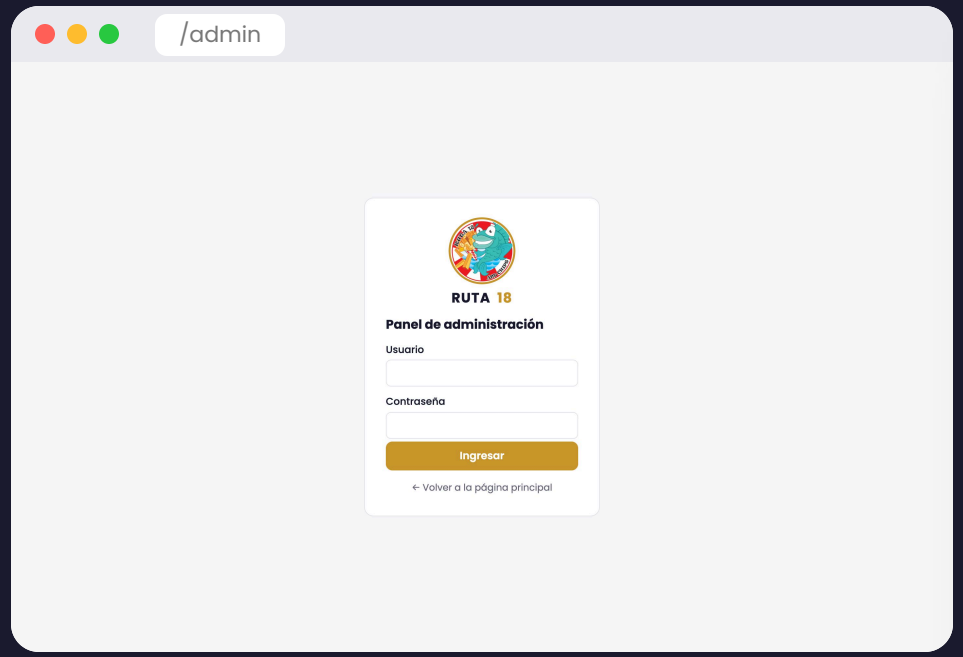

Si algo está mal, el campo se marca en rojo y aparece un aviso que explica el error.

Confirmación

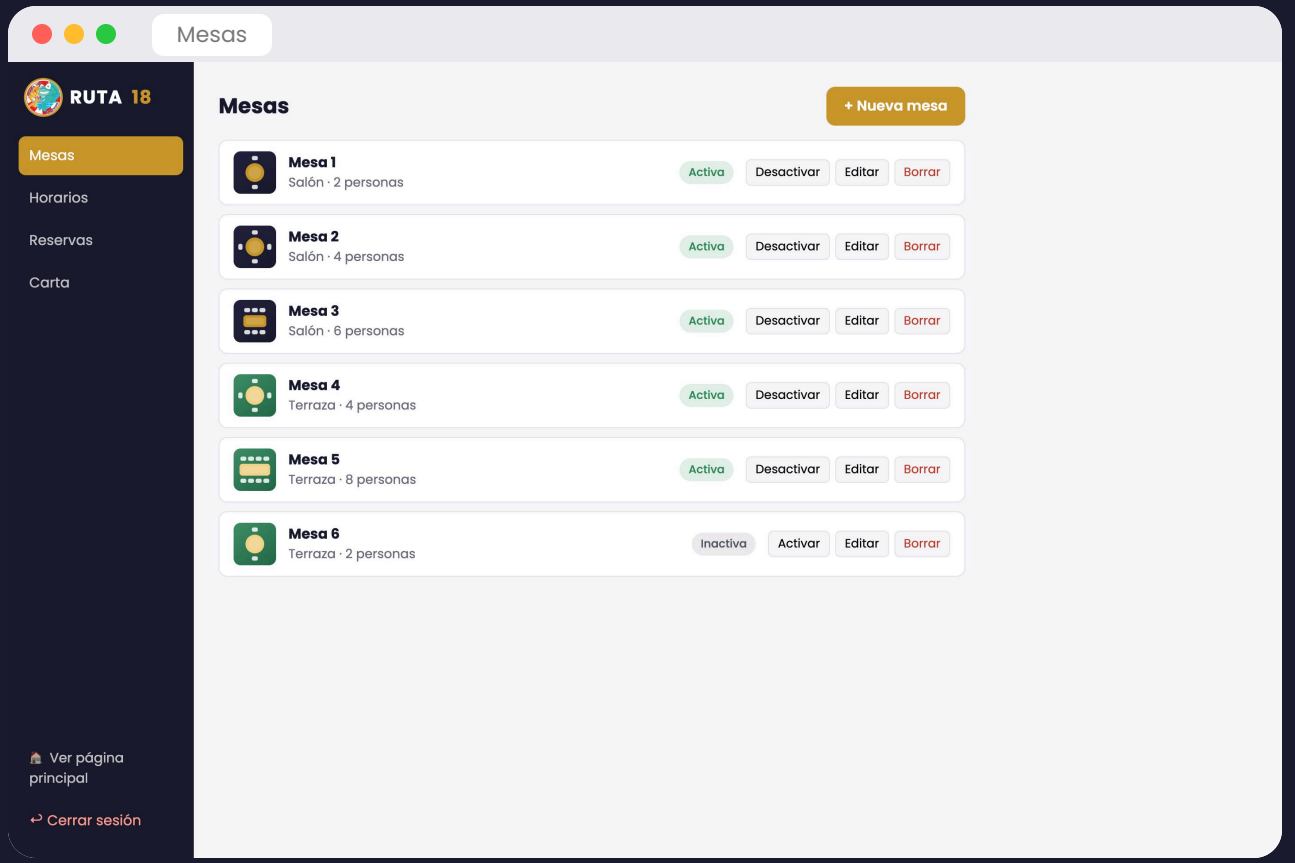



Comprobante con número de reserva, mesa, fecha, hora y personas (RF-06).

# Panel de administración

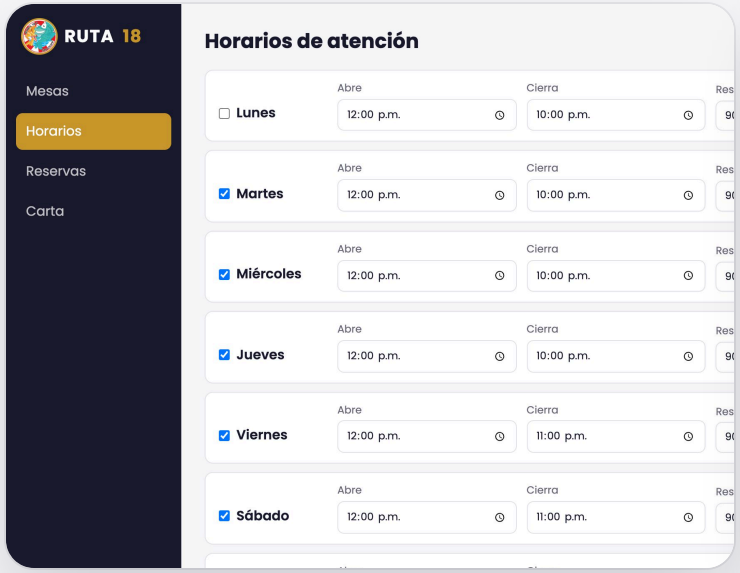



Acceso con usuario y contraseña (RF-08).

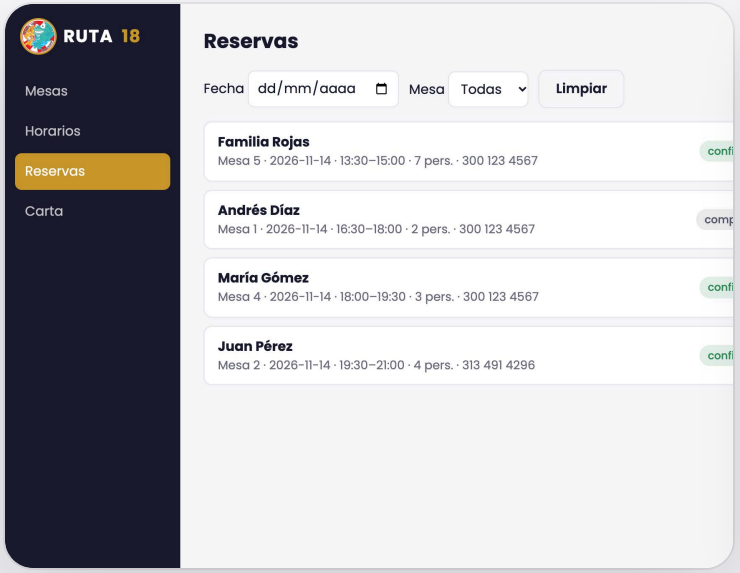



Registrar, editar, activar / desactivar y borrar mesas (RF-09, 10 y 13).

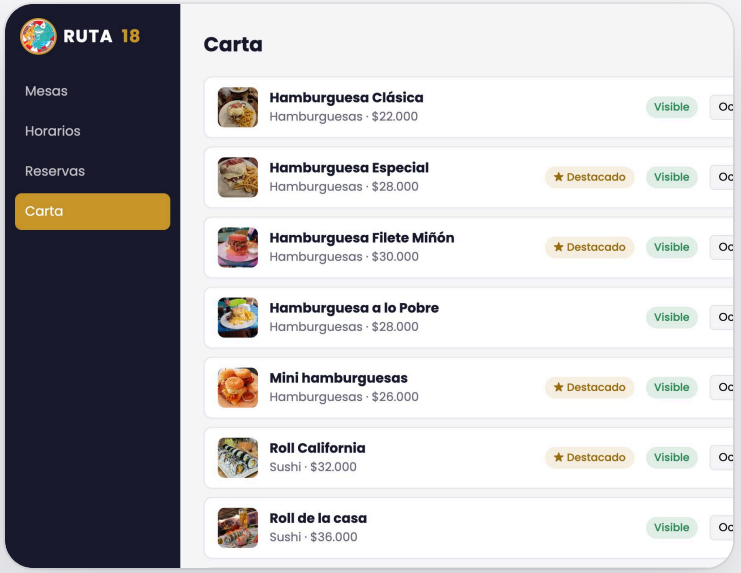
# Horarios, reservas y carta




**Horarios:** apertura, cierre y duración por día (RF-11).




**Reservas:** filtros por fecha y mesa, y cambio de estado (RF-12).

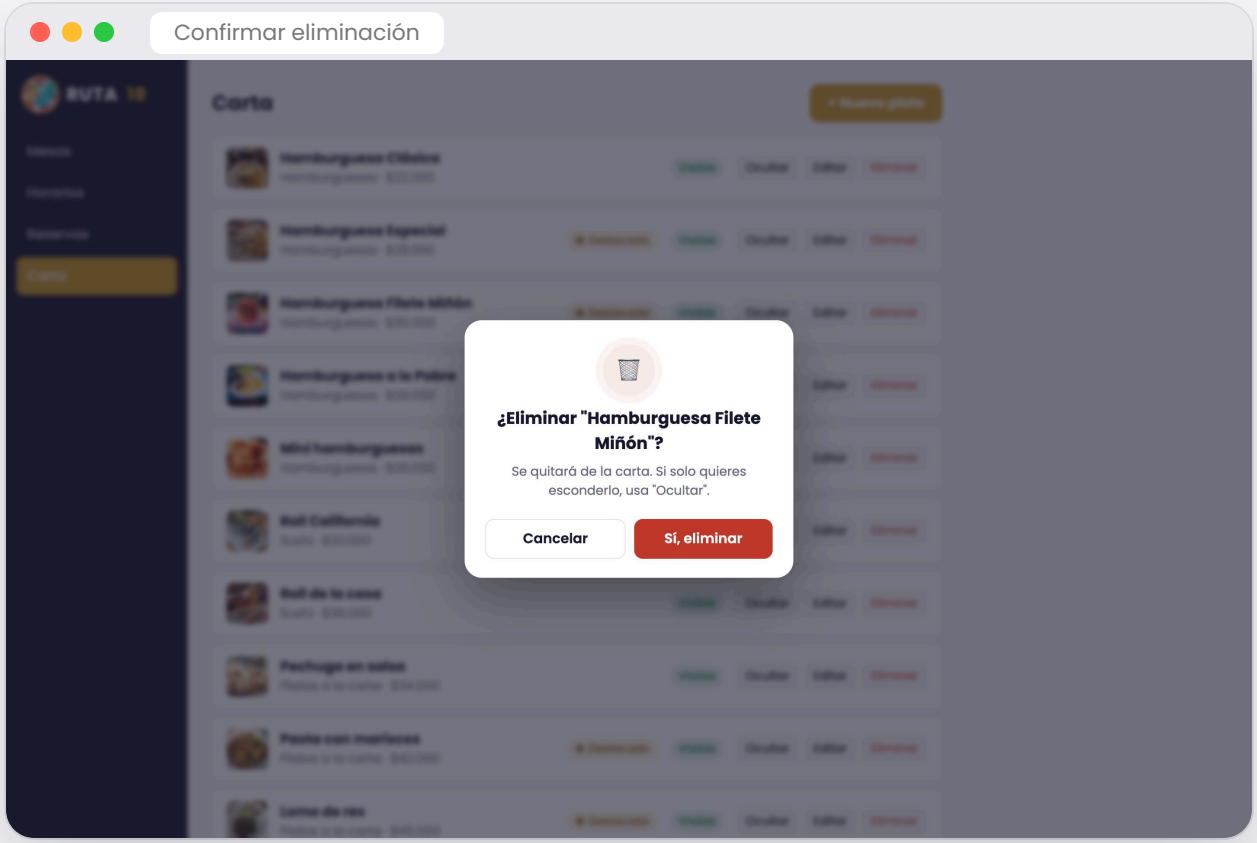



**Carta:** agregar, editar, ocultar, destacar y eliminar (RF-14).

📷 Al agregar un plato se puede **subir su foto** desde el computador; se reduce a 500 px y se guarda en el navegador.

🔄 Si un cliente reserva en otra pestaña, el panel **se actualiza solo** y avisa que llegó una reserva.

# Confirmaciones y avisos con la marca



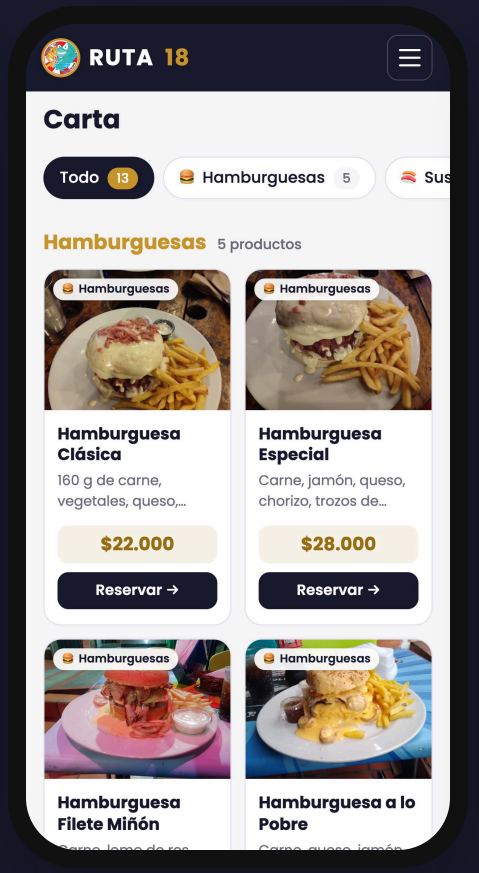

Se reemplazaron las ventanas grises del navegador (`alert` / `confirm`) por tarjetas propias.

- **Confirmación** antes de borrar o cerrar sesión, con botón rojo para acciones peligrosas.
- **Avisos** verdes al guardar y rojos ante errores; se cierran solos.
- **Campos marcados** en rojo donde está el problema.
- Se cierran con **Escape** o tocando fuera.
- Validaciones: obligatorios, capacidad, precios, horarios coherentes, teléfono, nombres de mesa repetidos, fechas y horas pasadas.

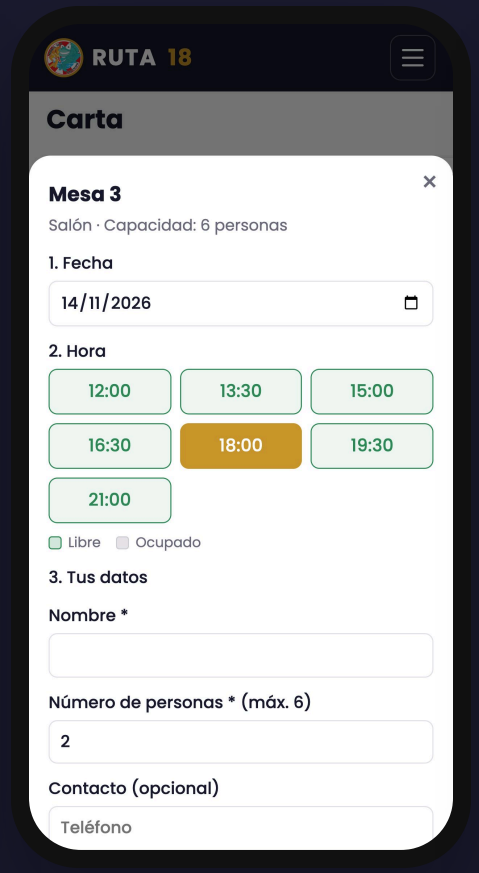
# Pensado para el celular



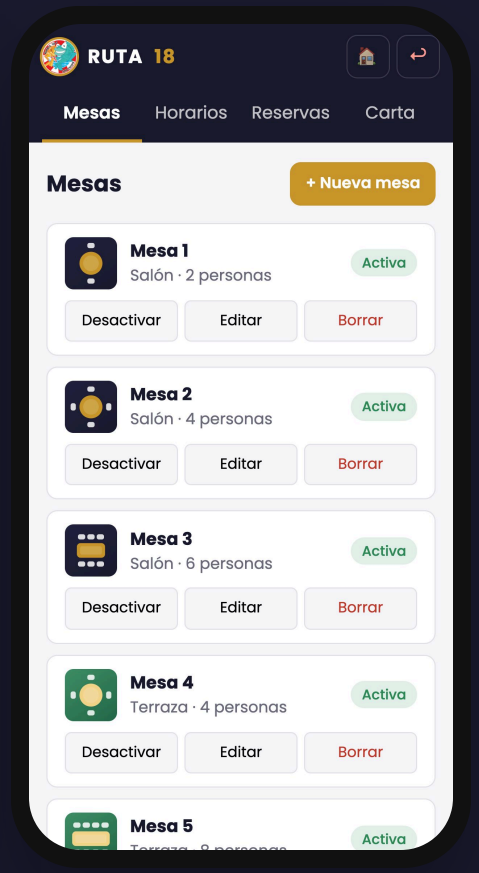

Menú hamburguesa




Carta en tarjetas




Reserva como hoja inferior




Admin con pestañas

# Estructura y publicación

```
ruta18-reservas-app/
├── index.html      # Página del cliente
├── admin.html      # Panel de administración
├── vercel.json     # URLs limpias (/admin)
├── css/
│   └── styles.css  # Estilos y responsivo
├── js/
│   ├── data.js     # Listas de datos
│   ├── ui.js       # Avisos, diálogos, íconos
│   ├── cliente.js  # Lógica del cliente
│   └── admin.js    # Lógica del admin
├── img/            # Logo y fotos de platos
└── README.md
```

1. El código se versiona con **Git** y se sube a **GitHub** con commits descriptivos.
2. En **vercel.com/new** se importa el repositorio (preset *Other*, sin build).
3. Vercel publica la app con **HTTPS**; el panel queda en `/admin`.
4. Cada `git push` a `main` vuelve a desplegar automáticamente.

# Pruebas realizadas

| ESCENARIO | RESULTADO ESPERADO | ESTADO |
|---|---|---|
| Reserva completa (mesa, fecha, hora, datos) | Comprobante y la hora queda ocupada | ✓ |
| Elegir una hora ya ocupada | Aviso "Horario ocupado" | ✓ |
| Más personas que la capacidad | Aviso con el máximo permitido | ✓ |
| Fecha u hora que ya pasó | No se puede reservar | ✓ |
| Día sin atención (lunes) | "El restaurante no atiende este día" | ✓ |
| Login con datos incorrectos / correctos | Aviso de error / entra al panel | ✓ |
| Desactivar una mesa · ocultar un plato | Desaparecen para el cliente | ✓ |
| Cancelar una reserva | La hora vuelve a quedar libre | ✓ |
| Horario con cierre antes de apertura | No se guarda y se indica el error | ✓ |
| Celular (320–390 px), tablet (768 px) y escritorio | Sin desbordes, todo legible | ✓ |

15 · PRÓXIMOS PASOS

# Mejoras futuras

### ☁️ Supabase

Base de datos compartida entre todos los dispositivos.

### 🔐 Autenticación real

Login seguro del administrador en el servidor.

### 💬 WhatsApp

Confirmación automática de la reserva al cliente.




# ¡Gracias!

Thania Daniela Quintana Garzon · Ruta 18 — Sistema de Reservas de Mesas